

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



**BÁO CÁO CUỐI KỲ
THỰC TẬP CƠ SỞ**

Đề tài: Xây dựng trang web thi online

Giảng viên hướng dẫn: TS. Kim Ngọc Bách

Họ và tên: Lê Xuân Thắng

Mã sinh viên: B23DCCN753

Hà Nội 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU CHUNG	3
1.1. Đặt vấn đề và lý do chọn đề tài.....	3
1.2. Mục tiêu dự án	3
1.3. Đối tượng và phạm vi hệ thống	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	3
2.1. Kiến trúc hệ thống tổng quan.....	3
2.2. Công nghệ phát triển phía Client (Frontend).....	4
2.3. Công nghệ phát triển phía Server (Backend).....	6
2.4. Hệ thống cơ sở dữ liệu và lưu trữ (Data Layer).....	6
2.5. Các dịch vụ tích hợp bên ngoài (Tùy chọn mở rộng).....	8
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	8
3.1. Phân tích yêu cầu hệ thống	8
3.1.1. Phân tích Actors	8
3.1.2. Phân tích Use Cases	8
3.1.3. Yêu cầu phi chức năng	9
3.2. Thiết kế luồng quy trình nghiệp vụ.....	10
3.2.1. Luồng tạo và quản lý đề thi.....	10
3.2.2. Luồng tổ chức và giám sát phiên kiểm tra thời gian thực.....	10
3.3. Thiết kế Cơ sở dữ liệu (Database Design).....	10
3.3.1. Lược đồ thực thể quan hệ.....	10
3.3.2. Các ràng buộc toàn vẹn dữ liệu.....	11
3.4. Thiết kế hệ thống API và Tương tác thời gian thực.....	11
3.4.1. Thiết kế REST API.....	11
3.4.2. Thiết kế luồng sự kiện WebSocket.....	12
3.5. Thiết kế Giao diện và Trải nghiệm Người dùng (UI/UX)	12
CHƯƠNG 4: KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM.....	13
4.1. Môi trường triển khai thực tế (Deployment Architecture)	13
4.2. Kết quả hiện thực hóa giao diện và tính năng.....	14
4.3. Kết quả xử lý rủi ro và tình huống ngoại lệ.....	17
4.4. Kết quả kiểm thử và đánh giá mã nguồn (Audit)	18

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	18
5.1. Kết quả đạt được	18
5.2. Các hạn chế còn tồn tại	19
5.3. Hướng phát triển tương lai.....	20

DANH MỤC HÌNH ẢNH

Hình 1. Kiến trúc hệ thống tổng quan	4
Hình 2. Hệ thống cơ sở dữ liệu	7
Hình 3. Sơ đồ Use Cases	9
Hình 4. Lược đồ thực thể quan hệ.....	11
Hình 5. Giao diện Dashboard.....	14
Hình 6. Giao diện tạo đề thi	15
Hình 7. Giao diện lobby	15
Hình 8. Giao diện Live Monitor.....	16
Hình 9. Giao diện Report	16
Hình 10. Giao diện tham gia phòng	17
Hình 11. Giao diện làm bài.....	17

CHƯƠNG 1: GIỚI THIỆU CHUNG

1.1. Đặt vấn đề và lý do chọn đề tài

Trong môi trường giáo dục hiện đại, việc đánh giá mức độ hiểu bài của học sinh ngay trên lớp là rất quan trọng. Tuy nhiên, các phương pháp kiểm tra miệng hay làm bài trên giấy truyền thống tiêu tốn nhiều thời gian chấm điểm và khó mang lại bức tranh tổng thể về tiến độ của cả lớp. Ngược lại, các hệ thống LMS (Learning Management System) lại quá cồng kềnh, đòi hỏi học sinh phải đăng ký tài khoản và tải ứng dụng phức tạp.

Từ thực tiễn đó, ClassPulse được phát triển với triết lý "Bảng trắng kỹ thuật số" — nhanh, sạch, và tập trung. Hệ thống cho phép giáo viên tạo và tổ chức bài kiểm tra nhanh chỉ trong vài thao tác, trong khi học sinh có thể tham gia ngay lập tức qua mã QR trên điện thoại cá nhân.

1.2. Mục tiêu dự án

Về mặt sư phạm: Giúp giáo viên thu thập dữ liệu thời gian thực về mức độ tiếp thu của học sinh, tự động hóa việc chấm điểm và xuất báo cáo phân tích chi tiết.

Về mặt trải nghiệm: Xây dựng trải nghiệm "Zero-friction" (không rào cản) cho học sinh: không tải app, không tạo tài khoản.

Về mặt kỹ thuật: Xây dựng một ứng dụng Single Page Application (SPA) với kết nối WebSocket hai chiều theo thời gian thực (Real-time), đảm bảo tính ổn định và khả năng tự động sao lưu dữ liệu.

1.3. Đối tượng và phạm vi hệ thống

Đối tượng sử dụng:

- Giáo viên: Người dùng chính, quản lý ngân hàng câu hỏi, mở phiên kiểm tra và theo dõi lớp học.
- Học sinh: Tham gia làm bài qua trình duyệt di động bằng mã phòng.

Phạm vi (MVP): Tập trung vào 3 loại câu hỏi (Trắc nghiệm, Đúng/Sai, Trả lời ngắn), hỗ trợ hiển thị công thức toán học (LaTeX) và bảng biểu (Markdown). Các tính năng AI (nhập câu hỏi từ PDF qua Gemini) và lưu trữ ảnh (Cloudinary) được tích hợp dưới dạng tùy chọn mở rộng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Kiến trúc hệ thống tổng quan

Hệ thống ClassPulse được thiết kế và triển khai dựa trên mô hình kiến trúc Client-Server hiện đại, tổ chức theo cấu trúc quản lý mã nguồn tập trung Monorepo. Mã nguồn được phân tách rõ ràng thành hai không gian làm việc (workspace) độc lập bao gồm phân

hệ Frontend (**apps/web**) chịu trách nhiệm về giao diện hiển thị và phân hệ Backend (**apps/server**) chịu trách nhiệm xử lý logic nghiệp vụ và lưu trữ dữ liệu.

CLIENT			
React 18 + Vite	Tailwind CSS	Zustand	Socket.io
MÔI TRƯỜNG TRUYỀN TẢI			
REST API (HTTP)		WebSocket (Socket.io)	
ỨNG DỤNG MÁY CHỦ			
Node.js + Express			
TẦNG DỮ LIỆU (DATA LAYER)			
PostgreSQL (Lưu trữ bền vững)		Redis (Lưu trữ tạm thời)	

Hình 1. Kiến trúc hệ thống tổng quan

Sự giao tiếp và đồng bộ hóa dữ liệu giữa Client và Server được thực hiện song song thông qua hai giao thức chính:

- **Giao thức REST API (HTTP):** Sử dụng cho các tác vụ CRUD (Create, Read, Update, Delete) truyền thông mang tính chất đồng bộ hoặc không yêu cầu phản hồi tức thời như: Đăng ký/đăng nhập tài khoản giáo viên, quản lý ngân hàng đề thi, cấu hình cài đặt phòng thi, truy xuất dữ liệu báo cáo lịch sử và nộp bài thi chính thức.

- **Giao thức WebSocket (Socket.io):** Sử dụng để thiết lập kênh truyền tải dữ liệu hai chiều (bidirectional) có độ trễ cực thấp và thời gian thực. Giao thức này phục vụ cho luồng nghiệp vụ tương tác trực tiếp trên lớp học bao gồm: Theo dõi danh sách học sinh tham gia phòng chờ (Lobby), phát tín hiệu bắt đầu/kết thúc làm bài, tự động lưu nháp câu trả lời (auto-save draft) và cập nhật tiến độ làm bài trực tiếp của lớp học lên màn hình giám sát của giáo viên.

2.2. Công nghệ phát triển phía Client (Frontend)

Phân hệ Frontend được xây dựng theo mô hình ứng dụng đơn trang SPA (Single Page Application) sử dụng ngôn ngữ mã nguồn mở TypeScript kết hợp với các thư viện tối ưu hóa hiệu năng hiển thị và quản lý trạng thái.

React 18: Thư viện mã nguồn mở giúp xây dựng giao diện người dùng dựa trên cấu trúc thành phần (Component-based) có khả năng tái sử dụng cao. Phiên bản React 18 mang lại cơ chế xử lý đồng thời (Concurrent Features), giúp tối ưu hóa luồng render giao diện, đảm bảo ứng dụng không bị hiện tượng giật lag khi nhận các luồng sự kiện real-time liên tục từ hệ thống WebSocket.

Vite: Công cụ hỗ trợ phát triển và đóng gói mã nguồn thế hệ mới, thay thế cho công cụ Build truyền thống như Create React App. Vite tận dụng tính năng Native ESM (ECMAScript Modules) của trình duyệt giúp tăng tốc độ khởi động môi trường lập trình và cơ chế HMR (Hot Module Replacement) cập nhật thay đổi giao diện tức thì.

TypeScript: Bản nâng cao của JavaScript bổ sung cơ chế kiểm lỗi tĩnh (Static Typing). Thành phần này đóng vai trò quan trọng trong việc định nghĩa chặt chẽ các kiểu dữ liệu cốt lõi của hệ thống (như cấu trúc câu hỏi MCQ, Đúng/Sai, Trả lời ngắn, dữ liệu phiên thi, báo cáo điểm số), giúp phát hiện sớm các lỗi cú pháp ngay trong quá trình lập trình.

Giải pháp quản lý trạng thái (State Management): Hệ thống phân tách rõ ràng hai loại trạng thái để áp dụng giải pháp quản lý tối ưu:

- **Trạng thái toàn cục của ứng dụng (Global State) với Zustand:** Thư viện quản lý trạng thái tối giản, có dung lượng siêu nhẹ. Zustand được áp dụng để lưu trữ các thông tin mang tính chất xuyên suốt phiên làm việc của Client như: Trạng thái đăng nhập của giáo viên (`authStore`), trạng thái kết nối trực tiếp của phòng thi (`sessionStore`) và tiến độ làm bài cục bộ của học sinh trên thiết bị di động (`studentStore`).

- **Trạng thái đồng bộ từ máy chủ (Server State) với TanStack React Query v5:** Quản lý toàn bộ dữ liệu được fetch từ REST API. Thư viện này tự động hóa việc lưu bộ nhớ đệm (caching), đồng bộ dữ liệu ngầm (background refetching) và tự động xóa bỏ dữ liệu lỗi thời (cache invalidation) ngay sau khi giáo viên thực hiện các hành động chỉnh sửa hoặc xóa đề thi, giúp giao diện luôn đồng nhất với cơ sở dữ liệu.

Tailwind CSS: Framework CSS theo tư duy tiện ích (Utility-first), cho phép lập trình viên áp dụng các class trực tiếp trên thẻ HTML. Tailwind CSS giúp tối ưu hóa dung lượng file CSS xuất bản (thông qua cơ chế Purge) và dễ dàng xây dựng giao diện đáp ứng (Responsive Design) cho cả màn hình máy chiếu của giáo viên (Desktop-First) lẫn màn hình điện thoại của học sinh (Mobile-First). Hệ thống kết hợp hàm tiện ích `cn` (phối hợp giữa `clsx` và `tailwind-merge`) để xử lý việc ghi đè các class CSS có điều kiện một cách an toàn.

Các công cụ hiển thị hỗ trợ:

- **KaTeX:** Thư viện toán học hiệu năng cao, thực hiện phân tích cú pháp và kết xuất (render) trực tiếp các chuỗi ký tự chứa mã toán học LaTeX (định dạng bởi ký tự vùng `$ ... $` hoặc `$$...$$`) thành mã HTML hiển thị trực quan ở tầng

presentation. Dịch vụ giúp ClassPulse đáp ứng tốt các câu hỏi thuộc khối ngành khoa học tự nhiên.

- **Markdown Table Parser:** Giao diện tích hợp bộ chuyển đổi định dạng bảng mã Markdown (GitHub-style tables) sang thẻ `<table>` của HTML, hỗ trợ hiển thị cấu trúc dữ liệu bảng biểu trực quan trên thiết bị di động mà không gây vỡ bố cục giao diện.

2.3. Công nghệ phát triển phía Server (Backend)

Máy chủ ứng dụng của ClassPulse được xây dựng dựa trên nền tảng hướng sự kiện bất đồng bộ, hướng tới mục tiêu xử lý lượng kết nối đồng thời lớn từ các lớp học.

Node.js & Express: Môi trường thực thi JavaScript phía máy chủ sử dụng mô hình I/O phi chặn (Non-blocking I/O) kết hợp vòng lặp sự kiện (Event Loop), tối ưu cho các ứng dụng trao đổi dữ liệu thời gian thực. Framework Express đóng vai trò định tuyến (routing) và quản lý hệ thống middleware cho các REST API, cấu trúc mã nguồn được phân chia theo kiến trúc Module hóa tách biệt (Auth, Quizzes, Sessions, Student, Reports, Upload) giúp dễ dàng bảo trì và nâng cấp.

Socket.io: Thư viện chuyên biệt xây dựng trên giao thức WebSocket, cung cấp giải pháp truyền tải dữ liệu hai chiều thời gian thực. Socket.io cung cấp các tính năng nâng cao như tự động kết nối lại (auto-reconnect), cơ chế fallback sang HTTP long-polling nếu mạng của học sinh bị chặn WebSocket, và khả năng quản lý phân phòng trực quan (`socket.join(roomCode)`) phục vụ việc cô lập luồng dữ liệu giữa các lớp học khác nhau.

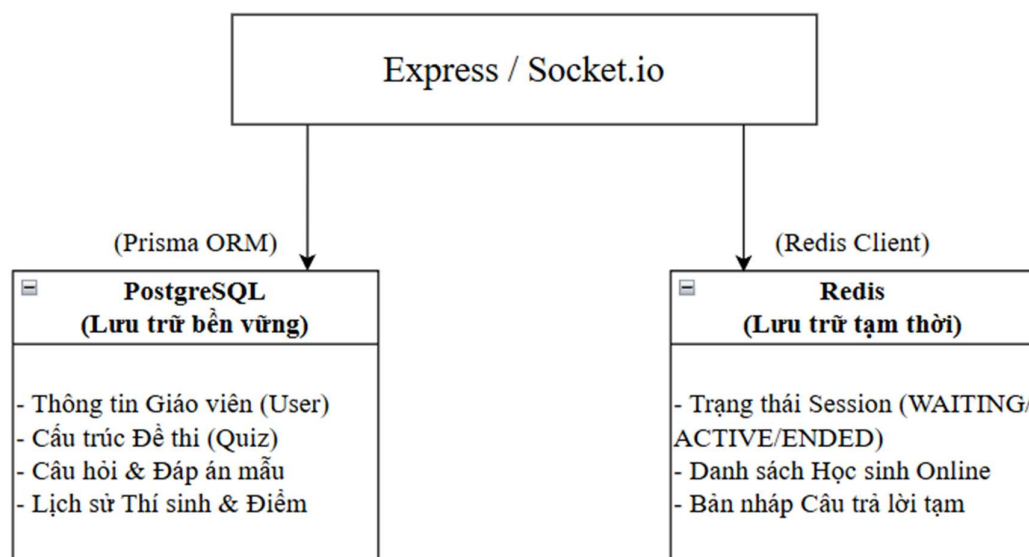
Cơ chế bảo mật và xác thực (Authentication & Authorization):

- **JSON Web Token (JWT):** Cơ chế xác thực phi trạng thái (Stateless Authentication) giúp giảm tải dung lượng lưu trữ phiên trên Server. Hệ thống sử dụng hai phân hệ JWT độc lập: Giáo viên sử dụng cặp Access Token ngắn hạn (15 phút) kết hợp Refresh Token bảo mật; học sinh tham gia lớp học không cần đăng ký tài khoản mà được hệ thống định danh thông qua một mã thông báo thí sinh ngắn hạn (`participantToken`, thời hạn 4 giờ) gắn chặt với một định danh `Session` cụ thể, đảm bảo học sinh không thể truy cập trái phép vào đề thi hoặc dữ liệu của phòng thi khác.

- **bcrypt:** Thư viện thực hiện băm mật khẩu (password hashing) một chiều kết hợp kỹ thuật thêm muối (salt), đảm bảo an toàn tuyệt đối cho thông tin mật khẩu của giáo viên khi lưu trữ trong cơ sở dữ liệu.

2.4. Hệ thống cơ sở dữ liệu và lưu trữ (Data Layer)

Hệ thống lưu trữ của ClassPulse phân định rõ ràng hai tầng dữ liệu: Tầng lưu trữ bền vững (Persistent Data) sử dụng cơ sở dữ liệu quan hệ và Tầng lưu trữ tạm thời tốc độ cao (Transient State) sử dụng in-memory database.



Hình 2. Hệ thống cơ sở dữ liệu

PostgreSQL 15.x & Prisma ORM:

- **PostgreSQL:** Hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, đáp ứng đầy đủ tiêu chuẩn ACID (Atomicity, Consistency, Isolation, Durability), bảo toàn tính chính xác của dữ liệu bằng điểm ngay cả khi có xung đột giao dịch dữ liệu.

- **Prisma ORM:** Bộ công cụ thiết lập ánh xạ thực thể quan hệ (ORM) thế hệ mới dành cho TypeScript. Prisma giúp quản lý các cấu trúc bảng phức tạp bao gồm **User**, **Quiz**, **Question**, **Option**, **Session**, **Participant**, **Answer** dưới dạng các đối tượng có kiểu dữ liệu tường minh. Lược đồ dữ liệu cấu hình thuộc tính **onDelete: Cascade** giữa mỗi quan hệ **Quiz -> Question -> Option**, tự động dọn dẹp các bản ghi phụ thuộc khi đề thi bị xóa, loại bỏ hoàn toàn hiện tượng rác dữ liệu.

Redis (Tàng đệm bộ nhớ): Cơ sở dữ liệu lưu trữ dưới dạng Key-Value hoạt động hoàn toàn trên bộ nhớ RAM, cung cấp tốc độ đọc/ghi dữ liệu trong thời gian dưới mili-giây. Trong hệ thống ClassPulse, Redis đóng vai trò trung tâm lưu trữ trạng thái trực tiếp của phòng thi nhằm tránh việc truy vấn quá tải vào PostgreSQL:

- **session:{sessionId}:state:** Lưu trữ siêu dữ liệu và trạng thái hiện hành của phiên kiểm tra (WAITING, ACTIVE, ENDED).

- **session:{sessionId}:students:** Quản lý tập hợp (Set) danh sách định danh của những học sinh đang duy trì trạng thái trực tuyến trong phòng thi.

- **session:{sessionId}:answers:** Lưu trữ một bảng băm (Hash) chứa toàn bộ các câu trả lời tạm thời của học sinh. Mỗi khi học sinh thay đổi đáp án trên điện thoại, dữ liệu được đồng bộ ngay vào cấu trúc Hash này qua WebSocket, đảm bảo bài làm không bị mất đi nếu thiết bị gặp sự cố ngắt kết nối mạng đột ngột.

Tất cả các khóa liên quan đến phiên thi trực tiếp trên Redis đều được cấu hình thời gian tự hủy (TTL) an toàn là 4 giờ.

2.5. Các dịch vụ tích hợp bên ngoài (Tùy chọn mở rộng)

Để gia tăng tính năng thông minh và tối ưu tài nguyên lưu trữ tệp tin của máy chủ ứng dụng, hệ thống tích hợp thêm hai giải pháp từ bên thứ ba dưới dạng các module có thể bật/tắt linh hoạt thông qua cấu hình biến môi trường:

- **Gemini AI SDK (@google/genai)**: Tích hợp mô hình ngôn ngữ lớn (LLM) của Google phục vụ luồng nghiệp vụ tạo đề thi tự động hỗ trợ giáo viên (AI-assisted PDF import). Khi giáo viên tải lên tài liệu PDF bài tập, Backend sử dụng cơ chế xử lý tệp trong bộ nhớ (In-memory buffer) qua `multer.memoryStorage()`, chuyển tiếp dữ liệu sang Gemini API bằng cấu hình ép buộc định dạng trả về dạng cấu trúc JSON nguyên khối. Mô hình AI thực hiện bóc tách nội dung câu hỏi, nhận diện đáp án đúng và trích xuất các công thức toán học dưới dạng chuỗi ký tự LaTeX để tự động điền vào biểu mẫu Quiz Builder, giúp giảm thời gian soạn đề thi xuống dưới 2 phút.

- **Clouinary SDK**: Giải pháp quản lý và lưu trữ tài nguyên đa phương tiện đám mây. Module này được gọi khi giáo viên đính kèm hình ảnh minh họa cho câu hỏi thi. Máy chủ Express đóng vai trò làm trạm trung chuyển dữ liệu dạng luồng (streamifier) chuyển buffer ảnh từ bộ nhớ RAM trực tiếp lên Clouinary API, lấy về liên kết URL tĩnh có giao thức bảo mật HTTPS để lưu vào trường dữ liệu imageUrl của bảng Question trong cơ sở dữ liệu PostgreSQL. Luồng xử lý này giúp giải phóng máy chủ khỏi việc lưu trữ file trực tiếp lên đĩa cục bộ, tối ưu hóa hiệu năng vận hành khi triển khai ứng dụng trên các môi trường đám mây.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

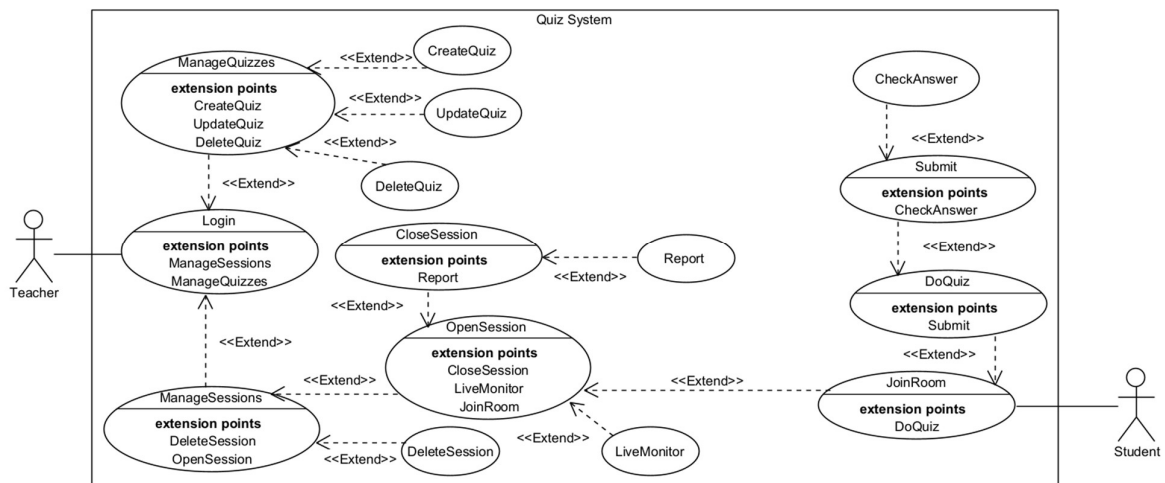
3.1.1. Phân tích Actors

Hệ thống ClassPulse xoay quanh hai nhóm tác nhân chính với các đặc điểm và quyền hạn hoàn toàn biệt lập:

- **Giáo viên (Primary User)**: Là người điều hành chính của hệ thống. Giáo viên cần đăng ký tài khoản và đăng nhập để truy cập vào phân hệ Teacher Portal. Tác nhân này có toàn quyền quản lý ngân hàng câu hỏi cá nhân, thiết lập các phiên kiểm tra và xem báo cáo dữ liệu lịch sử.

- **Học sinh (Secondary User)**: Là người tham gia bài kiểm tra. Đặc điểm cốt lõi của hệ thống là thiết kế luồng "Zero-friction", nghĩa là học sinh không cần tạo tài khoản, tải ứng dụng hay ghi nhớ mật khẩu. Tác nhân này tham gia hệ thống bằng cách truy cập trình duyệt web, nhập mã phòng hoặc quét mã QR do giáo viên cung cấp.

3.1.2. Phân tích Use Cases



Hình 3. Sơ đồ Use Cases

Các chức năng cốt lõi của hệ thống được chia theo vai trò người dùng như sau:

- Khối chức năng của Giáo viên:

- Quản lý tài khoản: Đăng ký, đăng nhập và xác thực phiên làm việc qua JSON Web Token (JWT).
- Quản lý đề thi (Quizzes): Tạo mới, chỉnh sửa, xóa đề thi (CRUD). Hỗ trợ tải ảnh minh họa lên Cloudinary hoặc dùng AI Gemini bóc tách câu hỏi từ file PDF.
- Quản lý phiên thi (Sessions): Mở phiên kiểm tra mới từ một đề thi, cấu hình thời gian làm bài, điều khiển vòng đời phiên thi (Bắt đầu, Kết thúc).
- Giám sát và Báo cáo: Theo dõi tiến độ học sinh theo thời gian thực (Live Monitor), xem báo cáo tổng quan sau phiên thi, phân tích độ khó từng câu hỏi và xuất bảng điểm.
- Quản lý lịch sử: Xem danh sách các phiên đã tổ chức, xóa phiên thi cũ để dọn dẹp dữ liệu.

- Khối chức năng của Học sinh:

- Tham gia phòng: Nhập mã phòng hợp lệ, điền tên và nhận phiên làm việc định danh tạm thời (participantToken).
- Làm bài thi: Trả lời câu hỏi (Trắc nghiệm, Đúng/Sai, Trả lời ngắn), hệ thống tự động lưu nháp dữ liệu.
- Nộp bài và xem kết quả: Xác nhận nộp bài cuối cùng, xem điểm số và đánh giá chi tiết nếu giáo viên cấu hình cho phép.

3.1.3. Yêu cầu phi chức năng

Bảo mật: Không thu thập thông tin định danh cá nhân của học sinh. Các API nghiệp vụ của giáo viên phải được bảo vệ nghiêm ngặt bằng JWT thông qua middleware phân quyền `role: TEACHER`. Mọi dữ liệu đầu vào phải được xác thực bằng schema `Zod` và chặn mã độc bằng cơ chế `sanitize-html`.

Trải nghiệm đa nền tảng: Giao diện học sinh bắt buộc tuân thủ nguyên tắc Mobile-first, không có thao tác cuộn ngang.

Độ tin cậy: Tính năng tự động lưu bài (Auto-save) phải hoạt động liên tục; trong trường hợp mất mạng, hệ thống phải giữ nguyên đáp án nhấp cho học sinh.

3.2. Thiết kế luồng quy trình nghiệp vụ

3.2.1. Luồng tạo và quản lý đề thi

1. Giáo viên truy cập màn hình Quiz Builder.
2. Nhập thông tin mô tả, thêm các câu hỏi hỗ trợ định dạng chữ (gồm văn bản, Markdown Table, công thức Toán học LaTeX KaTeX) hoặc đính kèm ảnh tải lên máy chủ Cloudinary.
3. Luồng tùy chọn: Giáo viên tải lên tệp PDF, hệ thống đẩy file tới Gemini API để trích xuất thành mảng JSON câu hỏi, sau đó đổ dữ liệu vào biểu mẫu trên giao diện.
4. Giáo viên nhấn Lưu, hệ thống xác thực đầu vào, loại bỏ thẻ HTML độc hại và ghi tuần tự vào các bảng `Quiz` -> `Question` -> `Option` trong PostgreSQL.

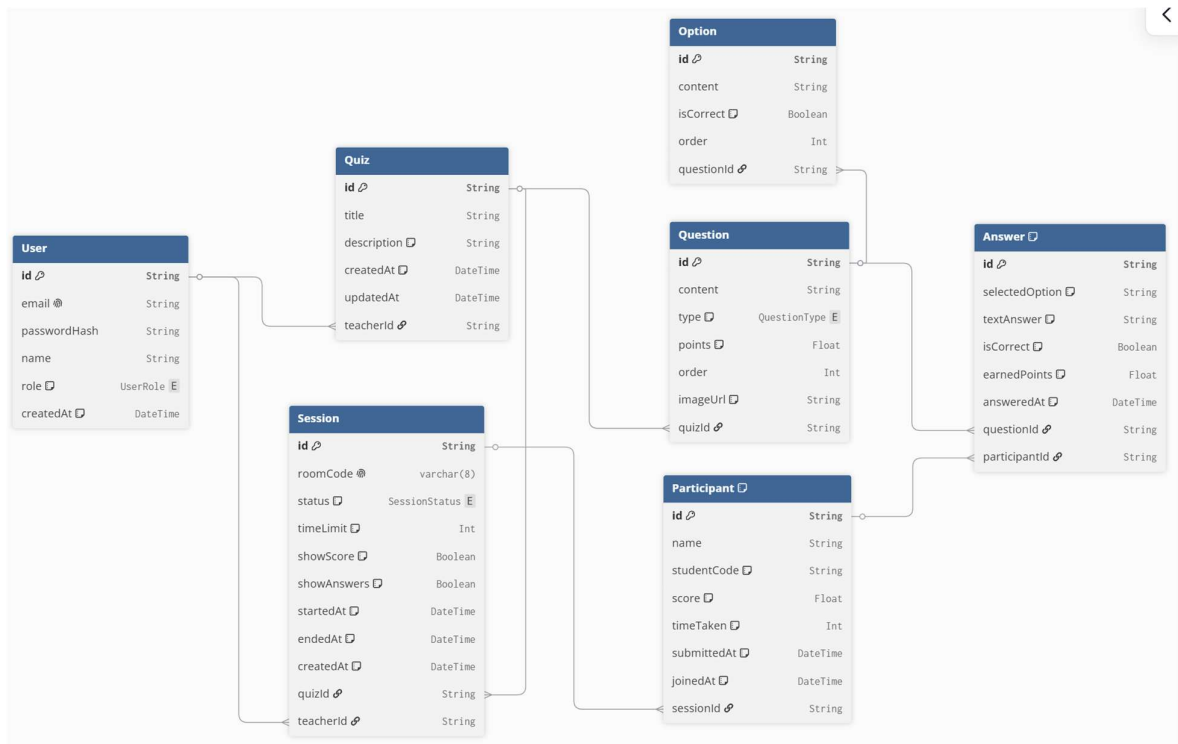
3.2.2. Luồng tổ chức và giám sát phiên kiểm tra thời gian thực

1. Giáo viên khởi tạo Session. Trạng thái phiên thi khởi điểm là WAITING. Màn hình trình chiếu mã QR hiển thị.
2. Học sinh quét mã QR, tham gia phòng thành công. Backend cấp mã `participantToken` và lưu định danh học sinh vào tập hợp (Set) trong bộ nhớ Redis.
3. Giáo viên nhấn "Bắt đầu làm bài". Backend cập nhật trạng thái Session thành ACTIVE và phát tín hiệu `session:started` qua Socket.io.
4. Khi học sinh chọn một đáp án, dữ liệu được truyền qua Socket.io. Backend lưu tạm vào Redis Hash `session:{sessionId}:answers` và phát tín hiệu `answer:received` ngược lại màn hình theo dõi Live Monitor của giáo viên.
5. Khi thời gian kết thúc hoặc giáo viên chủ động dừng, phiên thi chuyển sang trạng thái ENDED. Toàn bộ dữ liệu điểm số, thời gian làm bài từ Redis được hệ thống đối chiếu, chấm điểm tự động và ghi cố định (persist) vào bảng `Answer` của PostgreSQL.

3.3. Thiết kế Cơ sở dữ liệu (Database Design)

Hệ thống sử dụng Prisma ORM để thiết kế và quản lý lược đồ dữ liệu trên PostgreSQL. Sơ đồ dữ liệu được tối ưu hóa cho việc truy vấn báo cáo đa chiều.

3.3.1. Lược đồ thực thể quan hệ



Hình 4. Lược đồ thực thể quan hệ

3.3.2. Các ràng buộc toàn vẹn dữ liệu

Hệ thống xử lý chặt chẽ các trường hợp rác dữ liệu và mâu thuẫn báo cáo thông qua các ràng buộc nghiệp vụ:

- **Xóa nối tiếp (Cascade Delete):** Ràng buộc `onDelete: Cascade` được thiết lập từ `Quiz` xuống `Question` và từ `Question` xuống `Option`. Khi một đề thi bị xóa, các câu hỏi và đáp án mẫu liên quan sẽ tự động biến mất.

- **Chặn xóa đề thi lịch sử:** Nếu một `Quiz` đã được sử dụng để tạo ít nhất một `Session`, giáo viên không được phép sửa hoặc xóa `Quiz` đó nhằm bảo toàn tính toàn vẹn của các báo cáo phân tích cũ.

- **Ràng buộc duy nhất (Unique Constraints):** Thực thể `Participant` bị ràng buộc bởi cặp khóa `[sessionId, name]` để tránh việc hai học sinh nhập trùng tên trong cùng một phòng. Thực thể `Answer` bị ràng buộc bởi `[participantId, questionId]` để đảm bảo một học sinh chỉ có một đáp án cuối cùng ghi nhận cho mỗi câu hỏi.

- **Xóa dữ liệu phiên thi (Session Deletion):** Việc xóa một `Session` lịch sử sẽ thực thi quy trình xóa thủ công (Transaction) ngược từ `Answer` -> `Participant` -> `Session`, đồng thời kích hoạt bộ dọn dẹp các khóa trên Redis để tối ưu hóa bộ nhớ đệm.

3.4. Thiết kế hệ thống API và Tương tác thời gian thực

3.4.1. Thiết kế REST API

Giao diện lập trình ứng dụng RESTful được phân tách theo từng tiền tố tài nguyên (Resource prefix) tại `/api/v1`:

- `/auth/...`: Các endpoint xử lý đăng ký, đăng nhập và lấy thông tin phiên làm việc giáo viên (`/me`).
- `/quizzes/...`: Các endpoint CRUD bảo mật quản lý đề thi, và endpoint đặc biệt `POST /import-pdf` xử lý AI.
- `/sessions/...`: Các endpoint khởi tạo mã phòng ngẫu nhiên, cấu hình thời gian thi, và điều khiển đóng/mở phiên.
- `/reports/...`: Endpoint trích xuất kết quả thống kê dạng JSON chứa phân phối điểm, tỷ lệ câu đúng/sai để vẽ biểu đồ phân tích.
- `/join/...` và `/play/...`: Các endpoint công khai phục vụ luồng làm bài của học sinh. Yêu cầu truyền Header chứa `participantToken`.

Tất cả các API khi xảy ra lỗi đều phản hồi theo một chuẩn cấu trúc JSON duy nhất gồm `success: false` và đối tượng `error` chứa mã lỗi (error code) giúp Frontend hiển thị thông báo lỗi thân thiện.

3.4.2. Thiết kế luồng sự kiện WebSocket

Thay vì các request độc lập, hệ thống thiết lập các "Phòng" (Socket Rooms) mang tên `roomCode` để cô lập dữ liệu. Các luồng sự kiện (Events) được quy hoạch như sau:

- `student:joined` và `session:joined`: Sự kiện đồng bộ khi có học sinh quét QR thành công, tên học sinh sẽ nổi lên (pop-in) màn hình Lobby của giáo viên.
- `session:start` và `session:started`: Sự kiện giáo viên ra lệnh phát đề, chuyển màn hình điện thoại học sinh sang giao diện câu hỏi.
- `student:answer`, `answer:received`, và `answer:confirmed`: Sự kiện liên tục gửi đáp án nháp lên Server và cập nhật thanh tiến trình (Progress Bar) trên máy tính giáo viên mà không cần chờ nộp bài toàn bộ.
- `session:end`: Lệnh ép buộc thu bài đồng loạt nếu giáo viên muốn ngừng bài thi khẩn cấp.

3.5. Thiết kế Giao diện và Trải nghiệm Người dùng (UI/UX)

Hệ thống giao diện được thiết kế tuân thủ nghiêm ngặt triết lý "Bảng trắng kỹ thuật số" — tối giản, ít nút bấm thừa và tập trung trực tiếp vào dữ liệu.

Hệ thống phân cấp thiết bị:

- **Teacher Portal**: Thiết kế theo tư duy Desktop-first. Bảng điều khiển (Dashboard) sử dụng không gian lớn để bố trí biểu đồ thống kê (Recharts) và các thông số hoạt động. Màn hình chờ (Session Lobby) được tối ưu hóa đặc biệt với

mã phòng hiển thị bằng font chữ Monospace kích thước siêu lớn (text-4xl) và mã QR có độ tương phản cao, phục vụ việc trình chiếu bằng Projector trên bục giảng.

- **Student Interface:** Thiết kế Mobile-first tuyệt đối (max-width: 480px). Khu vực nội dung được đặt trọng tâm. Nút nộp bài, phân trang câu hỏi được bố trí vừa tầm tay trên màn hình cảm ứng.

Trạng thái tương tác và Hoạt ảnh (Animations): Mọi sự kiện thay đổi dữ liệu đều có phản hồi trực quan mà không yêu cầu tải lại trang (reload). Học sinh chọn đáp án sẽ thấy hiệu ứng viền sáng (border flash); đồng hồ đếm ngược chuyển màu đỏ cảnh báo khi thời gian dưới 30 giây; thông báo lỗi được hiển thị trực tiếp (Inline Error) dưới biểu mẫu thay vì dùng hộp thoại `alert()` gây gián đoạn trải nghiệm.

Hiển thị: Nội dung câu hỏi và đáp án phức tạp được bọc trong bộ kết xuất `RichTextDisplay` riêng biệt để bảo đảm các cấu trúc ma trận, căn bậc hai LaTeX hay bảng biểu Markdown được hiển thị chuẩn xác.

CHƯƠNG 4: KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM

4.1. Môi trường triển khai thực tế (Deployment Architecture)

Đưa hệ thống ClassPulse vào vận hành thực tế mà không phụ thuộc vào môi trường máy chủ cục bộ (Localhost), dự án đã áp dụng mô hình triển khai phân tán trên các nền tảng đám mây miễn phí (Platform as a Service - PaaS) tối ưu cho kiến trúc Monorepo. Cấu trúc triển khai được chia làm 4 thành phần chuyên biệt:

1. Dữ liệu cứng (Persistent Data) - Triển khai trên Neon: Cơ sở dữ liệu PostgreSQL được khởi tạo trên nền tảng Serverless Postgres của Neon. Đây là nơi lưu trữ toàn bộ dữ liệu mang tính lịch sử và vĩnh viễn của hệ thống bao gồm: thông tin tài khoản Giáo viên (User), cấu trúc đề thi (Quiz, Question, Option), và lịch sử làm bài (Session, Participant, Answer). Chuỗi kết nối được bảo mật và cung cấp cho Backend thông qua biến môi trường `DATABASE_URL`.

2. Dữ liệu mềm (Transient/Realtime Data) - Triển khai trên Upstash: Hệ thống Redis được triển khai trên Upstash Serverless Redis. Phân hệ này chịu trách nhiệm lưu trữ các dữ liệu "mềm" có vòng đời ngắn (được cấu hình TTL 4 giờ) như: trạng thái phòng thi trực tiếp (`session:{id}:state`), danh sách học sinh đang online (`session:{id}:students`), và đặc biệt là bản nháp câu trả lời theo thời gian thực (`session:{id}:answers`) để đảm bảo tốc độ phản hồi cực nhanh cho Socket.io. Chuỗi kết nối được cung cấp qua biến `REDIS_URL`.

3. Lõi xử lý nghiệp vụ (Backend) - Triển khai trên Render: Toàn bộ mã nguồn Node.js/Express và máy chủ Socket.io (`apps/server`) được đóng gói và triển khai trên nền tảng Render.com. Render đóng vai trò là API Gateway trung tâm, tiếp nhận cấu hình biến môi trường kết nối đến cả Neon và Upstash. Đồng thời, máy chủ được cấu hình giới

hạn chính sách chia sẻ tài nguyên nguồn gốc chéo (CORS) chỉ cho phép nhận các luồng yêu cầu (HTTP và WebSocket) đến từ đúng tên miền của Frontend, đảm bảo tính bảo mật.

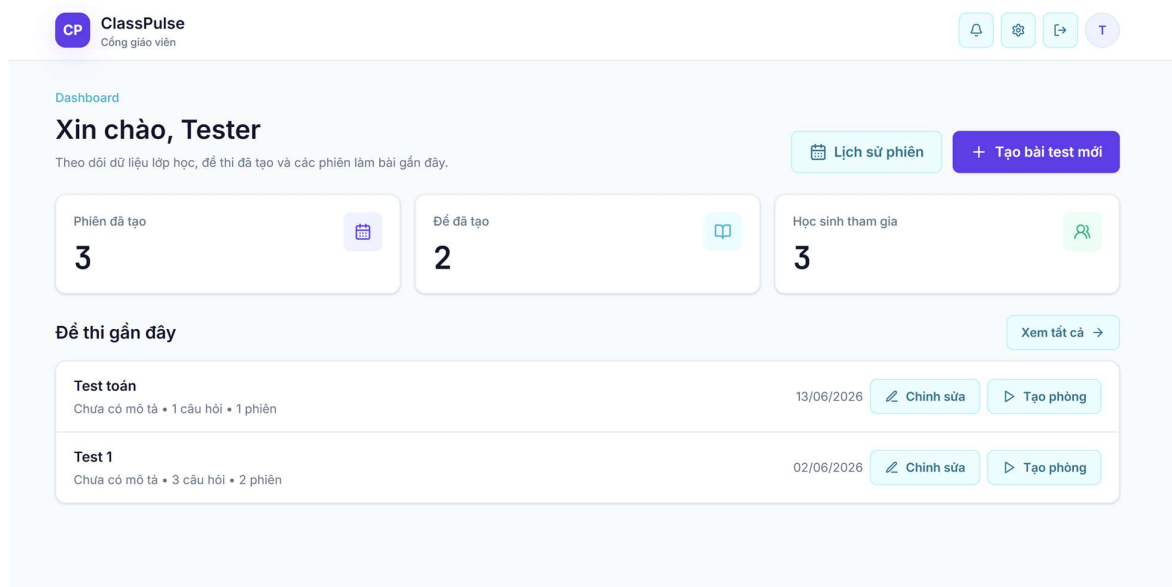
4. Giao diện người dùng (Frontend) - Triển khai trên Vercel: Ứng dụng Single Page Application (SPA) xây dựng bằng React/Vite (`apps/web`) được đóng gói thành các tệp tĩnh (HTML, CSS, JS) và lưu trữ trên mạng lưới CDN toàn cầu của Vercel. Ứng dụng client được liên kết với Backend thông qua hai biến môi trường `VITE_API_URL` (gọi REST API) và `VITE_WS_URL` (kết nối Socket.io) trở trực tiếp về tên miền của máy chủ Render.

4.2. Kết quả hiện thực hóa giao diện và tính năng

Quá trình cài đặt đã hiện thực hóa thành công các luồng giao diện chính tuân thủ theo nguyên tắc thiết kế đã đề ra:

Giao diện dành cho Giáo viên (Teacher Portal):

- **Bảng điều khiển (Dashboard):** Tích hợp thành công các biểu đồ và thẻ thống kê dữ liệu thực (tổng số đề, số phiên, học sinh) từ cơ sở dữ liệu. Giao diện tuân thủ nguyên tắc Desktop-first, hỗ trợ hiển thị tối ưu trên máy tính xách tay và máy tính bảng.



Hình 5. Giao diện Dashboard

- **Trình tạo đề thi (Quiz Builder):** Hỗ trợ giáo viên soạn thảo 3 loại câu hỏi (Trắc nghiệm, Đúng/Sai, Trả lời ngắn). Đặc biệt, hệ thống đã kết xuất thành công các định dạng phức tạp như bảng biểu (Markdown Table) và công thức toán học (KaTeX) trực tiếp trên trình duyệt.

Câu hỏi

1 câu trong đề thi này

Import từ PDF

Thêm câu hỏi

Câu 1

Nhập nội dung, loại câu hỏi và điểm số.

Xóa câu

Nội dung câu hỏi

Loại câu hỏi

Điểm

Câu 1: Cho hàm số $f(x) = \log_{\frac{1}{2}}(2x-1) + \int_0^x \frac{\sin(x)}{\sqrt{x^2+1}} dx$. Biết ma trận hệ số có dạng cực kỳ phức tạp như sau:

$$M = \begin{bmatrix} \frac{\alpha}{\sum_{i=1}^2 x_i} & \beta^2 & 0 \\ \int e^x dx & \lim_{x \rightarrow 0} \frac{\sin x}{x} & \infty \\ \frac{\sin x}{x} & \lim_{x \rightarrow 0} e^x & 1 \end{bmatrix}$$

Câu 1: Cho hàm số $f(x) = \log_{\frac{1}{2}}(2x-1) + \int_0^x \frac{\sin(x)}{\sqrt{x^2+1}} dx$. Biết ma trận hệ số có dạng cực kỳ phức tạp như sau:

$$M = \begin{bmatrix} \frac{\alpha}{\sum_{i=1}^2 x_i} & \beta^2 & 0 \\ \int e^x dx & \lim_{x \rightarrow 0} \frac{\sin x}{x} & \infty \\ \frac{\sin x}{x} & \lim_{x \rightarrow 0} e^x & 1 \end{bmatrix}$$

Tìm giá trị lớn nhất của hàm số trên khoảng $(0; +\infty)$.

Hình ảnh câu hỏi

PNG, JPG, JPEG hoặc WEBP. Tối đa 5MB.

Upload ảnh

Đáp án

Chọn chính xác một đáp án đúng.

Đúng

$2\log_{\frac{1}{2}}(5)$

$2\log_5(5)$

Đúng

$\pm \infty$

Thêm đáp án

Lưu thay đổi

Hình 6. Giao diện tạo đề thi

- Giám sát trực tiếp (Live Monitor) & Sảnh chờ (Lobby):** Màn hình sảnh chờ hiển thị mã QR và mã phòng kích thước lớn để thuận tiện trình chiếu. Màn hình giám sát trực tiếp cập nhật lưới trạng thái học sinh (đang làm, đã nộp) theo thời gian thực mà không cần làm mới trang.

Quay lại

Phòng chờ realtime

Test toán

Tùy chọn hiện tại

Không giới hạn · Cho xem điểm · Cho xem kết quả

Tùy chọn

Bắt đầu làm bài

MÃ PHÒNG

C9YP9U

<https://class-pulse-web-nu.vercel.app/join/C9YP9U>

Học sinh quét QR hoặc nhập mã phòng trên điện thoại để vào phòng chờ.

Học sinh đã vào

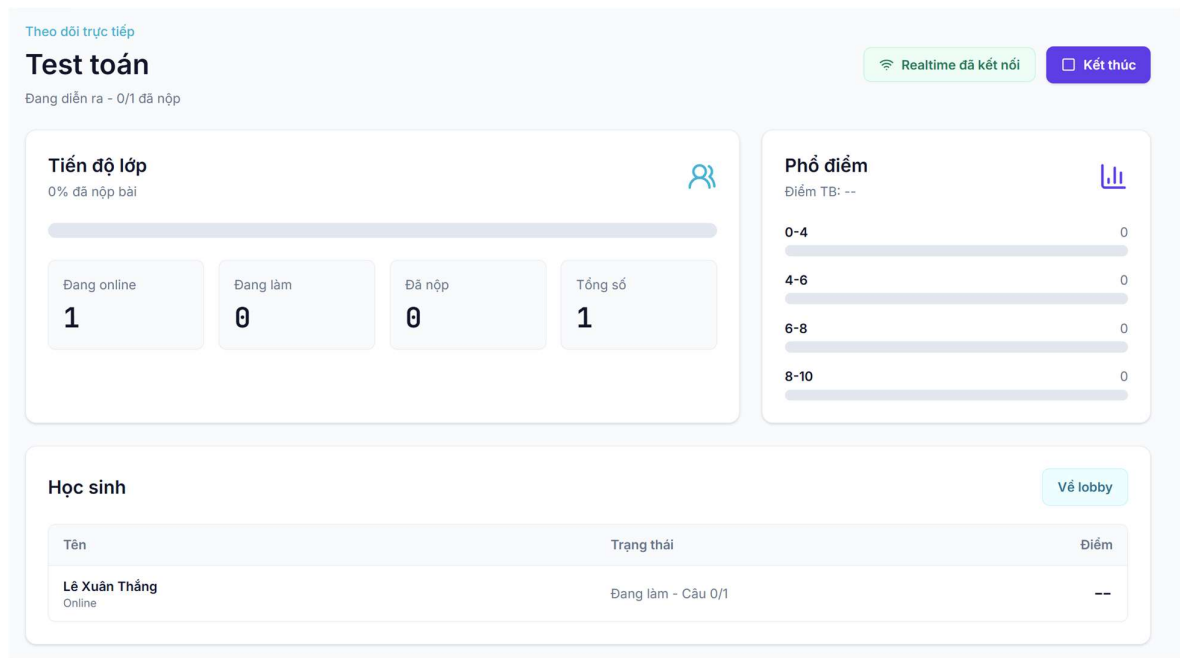
1/1 đang online

LX

Lê Xuân Thắng

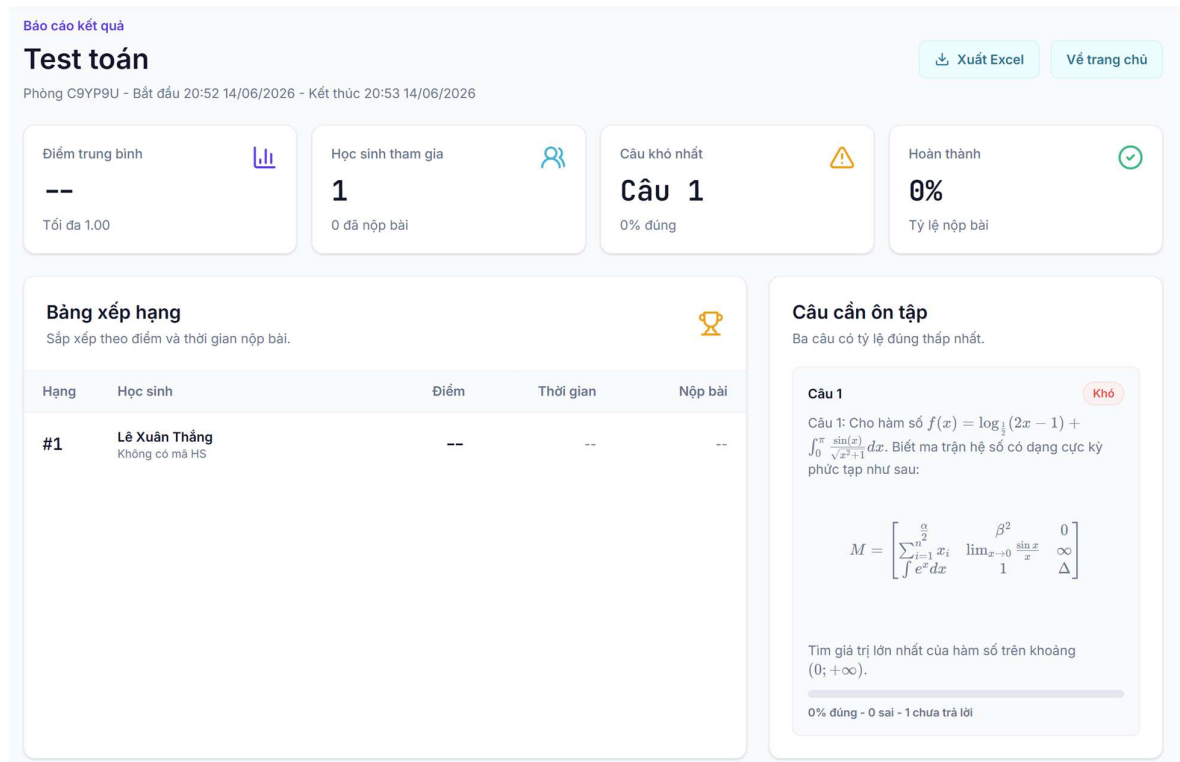
Đang chờ

Hình 7. Giao diện lobby



Hình 8. Giao diện Live Monitor

- **Báo cáo phân tích (Report):** Hiện thị bảng xếp hạng điểm số và tỷ lệ đúng/sai chi tiết cho từng câu hỏi, kèm theo tính năng xuất bảng điểm ra tệp Excel .xlsx thực thụ.



Hình 9. Giao diện Report

Giao diện dành cho Học sinh (Student Flow):

Thiết kế tuân thủ tuyệt đối quy tắc Mobile-first. Học sinh chỉ cần nhập mã phòng và tên để truy cập bài thi, không yêu cầu bất kỳ quy trình đăng ký tài khoản nào.

Hình 10. Giao diện tham gia phòng

Hình 11. Giao diện làm bài

Giao diện làm bài thi trực quan với bộ đếm ngược thời gian (chuyển sang trạng thái cảnh báo đỏ nhấp nháy khi còn dưới 30 giây). Các câu trả lời được tự động lưu nhấp ngay lập tức thông qua giao thức WebSocket.

4.3. Kết quả xử lý rủi ro và tình huống ngoại lệ

Hệ thống ghi nhận hoạt động ổn định và có khả năng tự phục hồi trong nhiều tình huống sự cố:

- **Xử lý mất kết nối mạng:** Cơ chế `GlobalErrorBoundary` và `OfflineIndicator` được tích hợp. Khi thiết bị mất mạng, màn hình hiển thị thông báo "Mất kết nối, đang tự động thử lại..." thay vì treo ứng dụng. Khi có mạng trở lại, ứng dụng tự động khôi phục luồng WebSocket và đồng bộ tiếp bản nhấp mà không làm mất dữ liệu bài làm của học sinh.

- **Bảo vệ tính toàn vẹn của báo cáo lịch sử:** Hệ thống ngăn chặn thành công việc xóa bỏ các đề thi (Quiz) đang được sử dụng trong các phiên kiểm tra lịch sử, tránh gây đứt gãy khóa ngoại (Foreign Key) trong cơ sở dữ liệu.

- **Quản lý vòng đời phiên thi (Session Lifecycle):** Nếu giáo viên kết thúc phiên hoặc rời khỏi phòng chờ, máy chủ gửi tín hiệu ép buộc (broadcast) đẩy toàn bộ học sinh ra khỏi phòng và chuyển hướng về trang tham gia ban đầu nhằm tránh lỗi "kẹt phòng" (stale room).

4.4. Kết quả kiểm thử và đánh giá mã nguồn (Audit)

Hệ thống đã trải qua các quy trình kiểm thử nghiêm ngặt trước khi đóng gói triển khai:

Kiểm định mã nguồn tĩnh (Static Analysis): Toàn bộ không gian làm việc (apps/web và apps/server) vượt qua quá trình kiểm tra cú pháp (npm run lint) và kiểm tra tính toàn vẹn kiểu dữ liệu (npm run typecheck) mà không phát sinh bất kỳ lỗi biên dịch nào. Lệnh npm audit xác nhận không có lỗ hổng bảo mật nghiêm trọng trong các gói thư viện sản xuất.

Kịch bản kiểm thử chức năng (Smoke Tests):

1. **Xác thực:** Luồng đăng ký, đăng nhập và cấp phát JWT cho giáo viên hoạt động chính xác. Middleware từ chối thành công (trả về lỗi HTTP 401) các luồng gọi API thiếu mã thông báo (token) hợp lệ.

2. **Toàn vẹn Dữ liệu:** Hoạt động thao tác (CRUD) đề thi với cơ sở dữ liệu Neon được ghi nhận lưu trữ thành công các mối quan hệ lồng nhau (Quiz -> Question -> Option) thông qua Prisma ORM.

3. **Độ trễ Thời gian thực (Realtime):** Khi học sinh bấm nộp bài hoặc chọn đáp án trên điện thoại qua Vercel, dữ liệu được truyền qua Render, lưu trữ tạm vào Upstash Redis và cập nhật lên màn hình giám sát của giáo viên với độ trễ (latency) không đáng kể.

4. **Chấm điểm tự động:** Logic chấm điểm hoạt động chính xác. Chấm điểm đạt chuẩn hóa văn bản đối với câu hỏi trả lời ngắn (Short Answer) và chấm điểm tỷ lệ lũy tiến đối với câu hỏi Đúng/Sai đa mệnh đề.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Hệ thống kiểm tra nhanh trực tuyến ClassPulse đã được phân tích, thiết kế và cài đặt thành công, đáp ứng tốt các mục tiêu ban đầu đặt ra trong việc số hóa quy trình đánh giá năng lực học sinh trên lớp. Qua quá trình thực hiện đồ án, dự án đã đạt được những kết quả nổi bật sau:

- **Về mặt Trải nghiệm người dùng (UX/UI):**

- Triển khai thành công triết lý "Zero-friction" (Không rào cản) dành cho đối tượng học sinh. Học sinh có thể tham gia phòng thi ngay lập tức thông qua mã QR hoặc mã phòng 6 ký tự mà không cần tạo tài khoản hay tải ứng dụng.

- Giao diện được thiết kế đáp ứng (Responsive) triệt để: Màn hình học sinh tuân thủ nguyên tắc Mobile-first gọn gàng, trong khi bảng điều khiển của giáo viên tuân thủ nguyên tắc Desktop-first với các biểu đồ trực quan, dễ đọc.

- Hệ thống hỗ trợ hiển thị mượt mà các định dạng văn bản phức tạp bao gồm công thức toán học (LaTeX qua KaTeX) và bảng biểu (Markdown Tables), đáp ứng tốt nhu cầu kiểm tra của khối ngành khoa học tự nhiên.

- Về mặt Kỹ thuật và Kiến trúc:

- Xây dựng thành công kiến trúc Monorepo phân tách rõ ràng Frontend (React 18, Vite) và Backend (Node.js, Express).

- Triển khai mô hình dữ liệu đa tầng hiệu quả: Sử dụng PostgreSQL (Prisma ORM) để lưu trữ dữ liệu lịch sử bền vững và sử dụng Redis để lưu trữ trạng thái phiên thi, danh sách học sinh trực tuyến cùng bản nháp câu trả lời theo thời gian thực.

- Tích hợp thành công giao thức WebSocket (Socket.io) giúp đồng bộ hóa dữ liệu hai chiều với độ trễ thấp, hỗ trợ tính năng tự động lưu bài (auto-save) và cập nhật tiến độ lớp học trực tiếp lên màn hình giáo viên.

- Về mặt Nghiệp vụ:

- Hoàn thiện luồng quản lý đề thi (CRUD) với cơ sở dữ liệu quan hệ, đi kèm cơ chế bảo vệ tính toàn vẹn của các đề thi đã từng được sử dụng.

- Xây dựng hệ thống tự động chấm điểm chính xác cho 3 loại câu hỏi: Trắc nghiệm (MCQ), Đúng/Sai (tính điểm theo tỷ lệ mệnh đề) và Trả lời ngắn (khớp chữ chuẩn hóa).

- Hệ thống báo cáo thống kê hoạt động ổn định, cho phép trích xuất bảng điểm Excel kèm theo các phân tích chuyên sâu về độ khó của từng câu hỏi.

5.2. Các hạn chế còn tồn tại

Bên cạnh những kết quả tích cực, do giới hạn về thời gian và tài nguyên triển khai của một dự án học tập, hệ thống vẫn còn một số điểm cần khắc phục:

- Hạn chế về khả năng mở rộng Real-time (Scaling): Máy chủ Socket.io hiện tại đang sử dụng bộ chuyển đổi lưu trữ trong bộ nhớ cục bộ (in-memory adapter) của một Node.js instance duy nhất. Mặc dù Redis đã được dùng để lưu trạng thái, hệ thống chưa cấu hình đầy đủ bộ chuyển đổi `@socket.io/redis-adapter` để hỗ trợ mở rộng ngang (horizontal scaling) khi lượng truy cập tăng vọt.

- Quy trình vận hành chưa tự động hóa: Hệ thống chưa được tích hợp các luồng CI/CD (Continuous Integration / Continuous Deployment) hoàn chỉnh, việc triển khai mã nguồn lên các môi trường đám mây vẫn cần sự can thiệp thủ công ở một số bước.

- **Giới hạn của module xuất báo cáo:** Tính năng tạo báo cáo định dạng PDF từ phía máy chủ (Server-side) chưa được hoàn thiện; hệ thống hiện tại mới chỉ hỗ trợ xuất dữ liệu thô ra tệp Excel từ phía Client.

- **Phụ thuộc vào dịch vụ bên thứ ba:** Các tính năng tải ảnh minh họa và dùng AI trích xuất câu hỏi từ PDF là các module mở rộng phụ thuộc vào API Key của Cloudinary và Gemini. Nếu không có cấu hình từ môi trường hoặc hết hạn ngạch (quota) miễn phí, hệ thống chỉ hỗ trợ trả về lỗi HTTP 503 (Graceful degradation) thay vì có giải pháp thay thế nội bộ.

5.3. Hướng phát triển tương lai

Dựa trên những hạn chế đã phân tích, lộ trình phát triển và nâng cấp hệ thống ClassPulse trong tương lai được định hướng như sau:

- **Tối ưu hóa kiến trúc hạ tầng:**

- Tích hợp chính thức `@socket.io/redis-adapter` vào máy chủ Backend để cho phép hệ thống chạy đa luồng, đa máy chủ (multi-instances), đảm bảo khả năng phục vụ hàng ngàn phòng thi cùng lúc.
- Cài đặt hệ thống hàng đợi công việc (Job Queue) thông qua thư viện BullMQ kết hợp với Redis để xử lý bất đồng bộ các tác vụ nặng, tiêu biểu như luồng trích xuất báo cáo PDF tự động ở phía máy chủ (Server-side).

- **Mở rộng tính năng nghiệp vụ:**

- Phân tích xu hướng học tập (Learning Trends): Bổ sung các biểu đồ đánh giá năng lực của học sinh xuyên suốt qua nhiều phiên kiểm tra khác nhau, giúp giáo viên nhận diện sớm những học sinh đang mất gốc kiến thức.
- Ngân hàng câu hỏi cộng tác (Question Bank): Phát triển tính năng cho phép các giáo viên trong cùng một trường hoặc tổ bộ môn chia sẻ, đóng góp và sử dụng chung các bộ đề thi.
- Yếu tố trò chơi hóa (Gamification): Tích hợp bảng xếp hạng thời gian thực (Leaderboard) hiển thị ngay trong quá trình làm bài và hệ thống huy hiệu (Badges) để tăng tính tương tác, cạnh tranh lành mạnh trong lớp học.
- Giám sát thi và chống gian lận: Nhằm nâng cao độ tin cậy cho bài kiểm tra, hệ thống sẽ được bổ sung cơ chế giám sát hành vi làm bài của học sinh.

- **Tích hợp và Đa nền tảng:**

- Xây dựng module kết nối qua API (LMS Integration) với các nền tảng giáo dục phổ biến như Google Classroom hay Moodle để đồng bộ danh sách lớp học và tự động chuyển điểm số.
- Nghiên cứu và phát triển phiên bản ứng dụng di động gốc (Native App) bằng công nghệ React Native dành riêng cho giáo viên, hỗ trợ quản lý lớp học mọi lúc mọi nơi.